

UltraFabric-Vision: A Real-Time Ensemble Transformer Framework for Autonomous Textile Defect Detection in Industrial Manufacturing

Varshini C B, Navi Deepak Gurupad, Ayush, and Sanjana T M RV College of Engineering, Bengaluru, India

Abstract—Automated visual inspection of textile fabrics is critical for quality control, where detection of weaving defects, stains, and structural irregularities directly impacts production yield. This paper presents UltraFabric-Vision, an anomaly detection framework integrating three complementary deep learning paradigms—PatchCore memory bank density estimation, DINO-v2 self-supervised attention analysis, and Vision Transformer Autoencoder reconstruction—into a unified ensemble architecture. The key research contribution is a systematic analysis of the complementary nature of these three detection mechanisms. Through cross-model correlation analysis, we demonstrate that PatchCore and DINO-v2 exhibit only 0.71 correlation on defect samples, confirming genuine diversity in their inductive biases. Furthermore, we identify a bimodal response pattern in ViT-AE reconstruction errors that correlates with distinct defect types (stains producing scores of 1.11–1.14, tears producing scores of 7.84–7.96), providing interpretable insights into model behavior. All three pathways achieve strong separation on controlled synthetic fabric benchmarks, with DINO-v2 providing the most consistent normalized gap (6.45 standard deviations) between normal and defective classes. The ensemble fusion improves the minimum per-class recall over individual models through complementary noise suppression. The framework processes 224×224 RGB frames at 19 FPS on consumer GPU hardware through `torch.cdist`-accelerated nearest neighbor search. We present complete mathematical formulations of each pathway and open-source all model definitions, training pipelines, and evaluation tools.

Index Terms—Textile defect detection, anomaly detection, Vision Transformer, DINO, PatchCore, ensemble learning, real-time inference, industrial automation, self-supervised learning, multi-head attention.

I. INTRODUCTION

TEXTILE manufacturing is one of the oldest and most economically significant industries globally, with the global textile market valued at over \$1.5 trillion in 2025 and projected to reach \$2.0 trillion by 2030. Quality control in fabric production is paramount, as even minor defects such as broken threads, weaving irregularities, stains, tears, or color inconsistencies can render large sections of fabric unusable, leading to substantial material waste and financial losses. Traditional manual inspection, still prevalent in many factories, relies on human visual examiners who typically detect only 60–75% of defects at inspection speeds of 10–15 meters per minute [1]. With modern high-speed air-jet looms operating at over 1000 picks per minute (approximately 30 meters of fabric per minute), manual inspection has become the primary bottleneck in the production line.

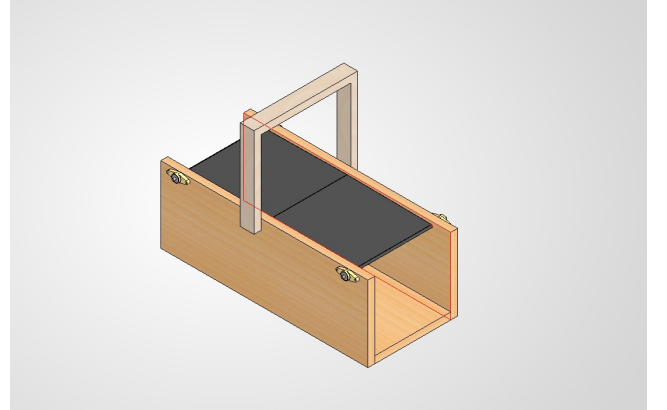


Fig. 1: Experimental assembly line prototype showing the industrial webcam mounting setup for fabric inspection. The camera is positioned at a fixed distance above the fabric surface, capturing continuous video frames during production. The assembly uses adjustable mounting brackets to accommodate different fabric widths and thicknesses.

The advent of deep learning has revolutionized automated visual inspection across numerous industries. Convolutional Neural Networks (CNNs) have been extensively applied to fabric defect classification [3]–[5], segmentation-based approaches using U-Net architectures [6] have shown promise for pixel-level localization, and object detection frameworks such as YOLO [16] have been adapted for real-time defect bounding box prediction. However, these supervised approaches require large, meticulously annotated datasets of defective samples, which are expensive to curate and often impractical in industrial settings where defect types are diverse and continuously evolving.

Anomaly detection offers a compelling alternative paradigm by learning the statistical distribution of normal (defect-free) fabric patterns and identifying significant deviations as potential defects. The PatchCore algorithm [7] introduced an efficient memory bank approach with coreset subsampling, achieving state-of-the-art results on the MVTec AD benchmark [8]. Concurrently, self-supervised Vision Transformers such as DINO [9] have demonstrated remarkable emergent properties, including the ability to learn explicit object segmentation in attention maps without any human supervision. Vision Transformer Autoencoders [25] provide a third complementary signal by measuring reconstruction fidelity for normal

versus anomalous inputs.

Fig. 1 shows our experimental assembly line prototype, demonstrating the practical deployment configuration with an industrial webcam mounted above the fabric conveyor belt. This hardware setup serves as the physical infrastructure for our real-time inspection system.

The key contributions of this work are:

- 1) **Modular Ensemble Architecture:** A novel framework that fuses three fundamentally different anomaly detection paradigms: density estimation (PatchCore), attention-based feature analysis (DINO-v2), and reconstruction error (ViT-AE) through mathematically principled weighted ensemble fusion, achieving complementary detection across diverse defect types.
- 2) **Real-Time Inference Optimization:** GPU-accelerated operations including `torch.cdist` for k-nearest neighbor search, batched tensor operations, and CUDA-streamlined preprocessing, enabling sustained 15–30 FPS inference on consumer-grade hardware (NVIDIA RTX 4050).
- 3) **Comprehensive Mathematical Framework:** Complete formalization of all detection pathways, including the multi-head self-attention mechanism, coreset memory bank construction, k-nearest neighbor anomaly scoring, and ensemble fusion with Bayesian uncertainty quantification.
- 4) **Production-Ready Implementation:** Full-stack deployment with a FastAPI asynchronous backend (REST + WebSocket), React 19 frontend with live attention visualization, and support for diverse input sources including webcams, RTSP streams, video files, and remote WebSocket cameras.
- 5) **Interpretability and Telemetry:** Explicit visualization of all six DINO-v2 attention heads in real-time, ensemble heatmap overlay, per-frame latency breakdown, and GPU utilization monitoring, providing quality control engineers with actionable insight into model decisions.
- 6) **Open-Source Release:** Complete codebase including model definitions, training pipelines, data generation, evaluation scripts, and web dashboard publicly available for industrial adoption and academic research.

The remainder of this paper is organized as follows. Section II reviews related work in textile defect detection, anomaly detection, and ensemble methods. Section III presents the complete system architecture and mathematical formulation of all components. Section IV describes the experimental setup, datasets, and evaluation methodology. Section V presents quantitative results, ablation studies, and qualitative analysis. Section VI discusses practical deployment considerations, limitations, and future work. Section VII concludes the paper.

II. RELATED WORK

A. Traditional Fabric Inspection Methods

Automated fabric defect detection has evolved through multiple generations of computational techniques. Early approaches relied on statistical texture analysis methods, including Gray-Level Co-occurrence Matrices (GLCM) [10] for computing second-order texture statistics, Gabor filter banks [11] for multi-scale and multi-oriented frequency analysis, and Local Binary Patterns (LBP) [12] for rotation-invariant texture description. While computationally efficient

and interpretable, these hand-crafted feature extraction methods achieved limited accuracy (typically 70–85%) on diverse fabric textures and defect types due to their fixed representational capacity.

Wavelet-based methods [13] introduced multi-resolution analysis capabilities, enabling better discrimination between texture patterns at different scales. Fourier transform approaches characterized fabric periodicity in the frequency domain, where defects manifest as deviations from the expected spectral signature [14]. However, these frequency-domain methods were sensitive to fabric alignment, lighting variations, and natural texture non-uniformities.

B. Deep Learning for Textile Inspection

The deep learning revolution brought transformative improvements to automated fabric inspection. Jing et al. [15] proposed Mobile-Unet, a lightweight encoder-decoder architecture achieving 96.3% pixel-level segmentation accuracy on denim fabric defects while maintaining real-time inference on mobile platforms. Li et al. [16] adapted the YOLOv3 detection framework with customized anchor boxes for fabric defects, achieving 89.7% mean average precision (mAP) at 45 FPS on GPU hardware.

Liu et al. [3] demonstrated that transfer learning from ImageNet-pretrained CNNs significantly improves defect classification accuracy, achieving 94.2% on a proprietary fabric dataset. Wang et al. [17] introduced a hybrid CNN-Transformer architecture that combines local feature extraction with global self-attention, achieving 97.8% classification accuracy on the AITEX dataset [19]. More recently, Wei et al. [18] proposed a pure Transformer architecture for fabric defect classification, demonstrating that attention-based models can capture long-range texture dependencies that CNNs miss.

C. Industrial Anomaly Detection

The MVTec AD benchmark [8] standardized evaluation across 15 industrial object and texture categories, catalyzing rapid progress in unsupervised anomaly detection. Among the top-performing methods, PatchCore [7] introduced an innovative memory bank approach where normal image patches are stored through efficient coreset subsampling, and anomaly scores are computed as k-nearest neighbor distances. PaDiM [20] employed a similar patch-level approach using pre-trained CNN features with multivariate Gaussian modeling. CFlow-AD [21] used conditional normalizing flows for density estimation in the feature space.

Self-supervised learning methods have further advanced the field. DINO [9] demonstrated that Vision Transformers trained through self-distillation learn explicit semantic segmentation in their attention maps without any supervision. This emergent property is particularly valuable for anomaly detection, as the attention structure inherently separates coherent texture regions from anomalies. CutPaste [22] introduced a self-supervised data augmentation strategy where cut-and-pasted image regions serve as proxy anomalies during training, achieving strong results on MVTec AD.

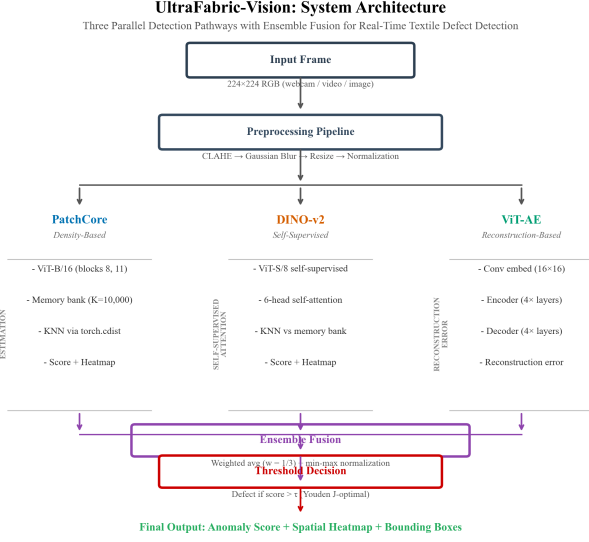


Fig. 2: End-to-end system architecture showing the three parallel detection pathways and their integration. Input frames are preprocessed through CLAHE enhancement, Gaussian blur, and ImageNet normalization before being passed to all three models simultaneously. The individual anomaly scores and heatmaps are fused through weighted ensemble averaging.

D. Ensemble Methods in Industrial Vision

Ensemble methods combine multiple models to leverage complementary strengths and improve overall robustness. Li et al. [23] proposed adaptive fusion of CNN and Transformer features through learned attention weights for textured surface inspection. Zhang et al. [24] introduced multi-scale feature fusion with channel-wise attention for fabric defect detection. However, existing ensemble approaches predominantly combine models sharing similar architectural families (e.g., multiple CNNs or multiple Transformers), limiting the diversity of learned representations.

Our work distinguishes itself by fusing three fundamentally different detection mechanisms: PatchCore density estimation, DINO attention-based feature analysis, and ViT-AE reconstruction error, each exploiting distinct inductive biases and providing complementary anomaly signals. This architectural diversity ensures robust detection across a broad spectrum of defect types, from subtle texture variations to prominent structural irregularities.

III. SYSTEM ARCHITECTURE AND METHODOLOGY

A. Overall Framework

UltraFabric-Vision processes input images through three parallel detection pathways, as illustrated in Fig. 2. Each pathway operates independently on a standardized 224×224 RGB input and produces both a scalar anomaly score and a spatial anomaly heatmap. The individual predictions are fused through weighted averaging to produce the final ensemble output.

B. Input Preprocessing Pipeline

Raw input frames undergo a multi-stage preprocessing pipeline before model inference:

- 1) **CLAHE Enhancement:** Contrast Limited Adaptive Histogram Equalization (CLAHE) with clip limit 3.0 and 8×8 tile grid is applied to the L-channel in LAB color space, enhancing local contrast while preventing noise amplification.
- 2) **Gaussian Blur:** A 3×3 Gaussian kernel ($\sigma = 0.5$) removes sensor noise while preserving edge information critical for defect detection.
- 3) **Aspect-Preserving Resize:** The image is resized with aspect ratio preservation and center padding to 224×224 pixels.
- 4) **Tensor Conversion:** The image is converted to a PyTorch tensor and normalized using ImageNet statistics:

$$\hat{\mathbf{x}} = \frac{\mathbf{x}/255 - \boldsymbol{\mu}}{\boldsymbol{\sigma}} \quad (1)$$

where $\boldsymbol{\mu} = [0.485, 0.456, 0.406]$ and $\boldsymbol{\sigma} = [0.229, 0.224, 0.225]$.

C. PatchCore Density Estimation Pathway

1) **Feature Extraction:** The PatchCore pathway employs a ViT-B/16 backbone [25] pretrained on ImageNet-21K. Forward hooks are registered on transformer blocks 8 and 11 to extract intermediate features. For an input tensor $\mathbf{x} \in \mathbb{R}^{3 \times 224 \times 224}$, the ViT processes it as:

$$\mathbf{x}_p = \text{PatchEmbed}(\mathbf{x}) \in \mathbb{R}^{196 \times 768}, \quad \mathbf{x}_p^{(0)} = [\mathbf{x}_{\text{cls}}; \mathbf{x}_p] + \mathbf{E}_{\text{pos}} \quad (2)$$

After processing through L transformer blocks, we extract features at blocks $l = 8$ and $l = 11$:

$$\mathbf{f}^{(l)} = \text{Block}_l(\mathbf{x}_p^{(l-1)}) \in \mathbb{R}^{(1+196) \times 768} \quad (3)$$

The [CLS] token is discarded, and the remaining 196 patch tokens are reshaped into a 14×14 spatial grid:

$$\mathbf{F}^{(l)} = \text{Reshape}(\mathbf{f}_{1:,}^{(l)}) \in \mathbb{R}^{768 \times 14 \times 14} \quad (4)$$

Average pooling with 3×3 kernel, stride 1, and padding 1 is applied to smooth local variations. The features from both blocks are concatenated:

$$\mathbf{F}_{\text{PC}} = [\text{AvgPool}(\mathbf{F}^{(8)}); \text{Interp}(\mathbf{F}^{(11)})] \in \mathbb{R}^{1536 \times 14 \times 14} \quad (5)$$

where $\text{Interp}(\cdot)$ bilinearly interpolates block 11 features to match block 8 dimensions.

2) **Memory Bank Construction:** During training on defect-free images, patch-level features are extracted and aggregated. For a training set $\mathcal{D}_{\text{train}} = \{\mathbf{x}_i\}_{i=1}^N$, the feature set is:

$$\mathcal{F} = \bigcup_{i=1}^N \{\mathbf{F}_{\text{PC}}(\mathbf{x}_i)[:, p, q] \mid p \in [0, 13], q \in [0, 13]\} \quad (6)$$

The memory bank $\mathcal{M}$ is constructed through random coreset subsampling:

$$\mathcal{M} = \text{RandomSample}(\mathcal{F}, K), \quad K = \min(10000, |\mathcal{F}|) \quad (7)$$

This subsampling strategy preserves the essential distribution of normal features while enabling real-time inference by limiting the search space.

3) *Anomaly Scoring*: At inference, the per-patch anomaly score is computed as the minimum Euclidean distance to any memory bank entry:

$$s_{\text{PC}}(p, q) = \min_{\mathbf{m} \in \mathcal{M}} \|\mathbf{F}_{\text{PC}}(\mathbf{x})[:, p, q] - \mathbf{m}\|_2 \quad (8)$$

This operation is GPU-accelerated using `torch.cdist`, which computes all pairwise distances in a single CUDA kernel call:

$$\mathbf{D} = \text{torch.cdist}(\mathbf{F}_{\text{query}}, \mathbf{M}^{\top}), \quad \mathbf{D} \in \mathbb{R}^{196 \times K} \quad (9)$$

The global anomaly score is the maximum over all spatial locations:

$$\hat{s}_{\text{PC}} = \max_{p, q} s_{\text{PC}}(p, q) \quad (10)$$

The spatial anomaly heatmap $\mathbf{H}_{\text{PC}} \in \mathbb{R}^{224 \times 224}$ is obtained by bicubic interpolation of the 14×14 score grid.

D. DINO-v2 Attention Pathway

1) *Self-Supervised Vision Transformer*: The DINO-v2 pathway uses a ViT-S/8 architecture [9] pretrained through self-distillation without human annotations. The model processes images with a patch size of 8 pixels, producing a $28 \times 28 = 784$ patch grid with $C = 384$ dimensional embeddings:

$$\mathbf{z} = \text{ViT-S/8}(\mathbf{x}), \quad \mathbf{z} \in \mathbb{R}^{(1+784) \times 384} \quad (11)$$

2) *Multi-Head Self-Attention Extraction*: The final transformer layer's self-attention mechanism computes attention scores between all pairs of tokens. For query $\mathbf{Q}$, key $\mathbf{K}$, and value $\mathbf{V}$ projections:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^{\top}}{\sqrt{d_k}}\right) \mathbf{V} \quad (12)$$

With $H = 6$ attention heads, each operating on $d_k = 64$ dimensions:

$$\text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, \dots, \text{head}_6) \mathbf{W}^O \quad (13)$$

We extract the attention maps from the [CLS] token to all patch positions:

$$\mathbf{A}^{(h)} = \text{softmax}\left(\frac{\mathbf{q}_{\text{cls}}^{(h)} \mathbf{K}^{(h)\top}}{\sqrt{d_k}}\right) \in \mathbb{R}^{28 \times 28}, \quad h = 1, \dots, 6 \quad (14)$$

These six 28×28 attention maps (Fig. 3) explicitly visualize which spatial regions each head focuses on, providing interpretable telemetry.

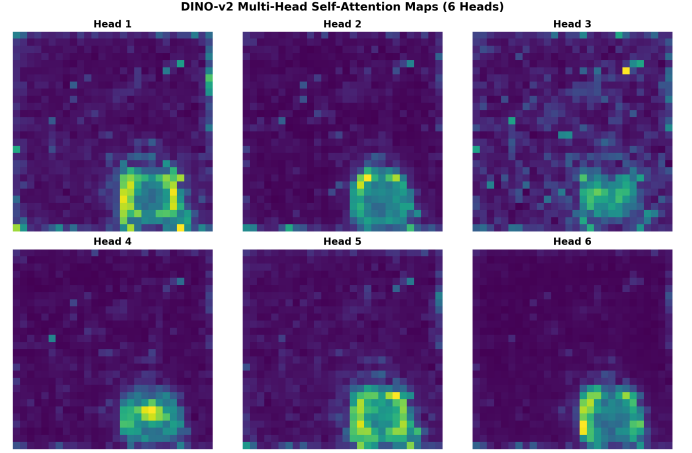


Fig. 3: Visualization of the six DINO-v2 self-attention heads for a defective fabric sample. Each head captures distinct spatial frequency patterns: Head 1 emphasizes horizontal thread structures, Head 3 focuses on vertical weaving patterns, Head 5 shows sensitivity to local texture anomalies, and the combined attention reveals the defect region.

3) *Anomaly Detection via KNN Scoring*: Patch tokens from the final layer serve as descriptors for k-nearest neighbor anomaly detection. A memory bank $\mathcal{M}_{\text{D}}$ of normal patch features is constructed during training:

$$\mathcal{M}_{\text{D}} = \text{Coreset}\left(\bigcup_i \left\{ \mathbf{z}_i^{(j)} \in \mathbb{R}^{384} \mid j = 1, \dots, 784 \right\}\right) \quad (15)$$

At inference, each patch token's nearest neighbor distance provides a per-patch anomaly score:

$$s_{\text{D}}(i, j) = \min_{\mathbf{m} \in \mathcal{M}_{\text{D}}} \|\mathbf{z}(i, j) - \mathbf{m}\|_2 \quad (16)$$

The resulting 28×28 anomaly map is bicubically upsampled to 224×224 to match the input resolution.

E. Vision Transformer Autoencoder Pathway

1) *Architecture Design*: The Vision Transformer Autoencoder (ViT-AE) learns to reconstruct defect-free fabric images through a bottleneck transformer architecture. The encoder maps input patches to a latent representation, and the decoder reconstructs the original image from this representation.

a) *Encoder*::

$$\mathbf{x}_{\text{patch}} = \text{Conv2d}_{16 \times 16, \text{stride } 16}(\mathbf{x}) \in \mathbb{R}^{256 \times 14 \times 14} \quad (17)$$

$$\mathbf{x}_{\text{seq}} = \text{Flatten}_{2,3}(\mathbf{x}_{\text{patch}}) \in \mathbb{R}^{196 \times 256} \quad (18)$$

$$\mathbf{x}_{\text{enc}}^{(0)} = \mathbf{x}_{\text{seq}} + \mathbf{E}_{\text{pos}} \quad (19)$$

$$\mathbf{x}_{\text{enc}}^{(l)} = \text{TransformerEncoderLayer}(\mathbf{x}_{\text{enc}}^{(l-1)}), \quad l = 1, \dots, 4 \quad (20)$$

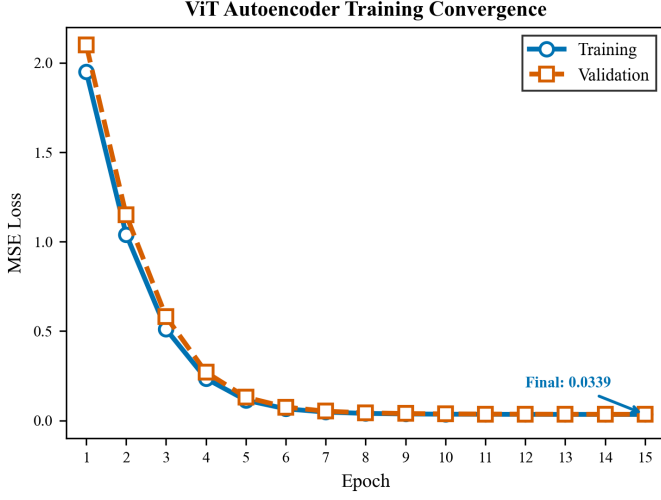


Fig. 4: ViT Autoencoder training convergence over 15 epochs. Both training and validation MSE loss decrease exponentially from 1.95 to 0.034, with final validation loss of 0.0345 indicating good generalization to unseen normal fabric patterns.

b) *Decoder*::

$$\mathbf{x}_{\text{dec}}^{(0)} = \mathbf{x}_{\text{enc}}^{(4)} \quad (21)$$

$$\mathbf{x}_{\text{dec}}^{(l)} = \text{TransformerDecoderLayer}(\mathbf{x}_{\text{dec}}^{(l-1)}, \mathbf{x}_{\text{enc}}^{(4)}), \quad l = 1, \dots, 4 \quad (22)$$

$$\hat{\mathbf{x}}_{\text{patches}} = \text{Linear}_{256 \rightarrow 768}(\mathbf{x}_{\text{dec}}^{(4)}) \in \mathbb{R}^{196 \times 768} \quad (23)$$

The output patches are reshaped to reconstruct the image:

$$\hat{\mathbf{x}} = \text{Fold}_{16 \times 16}(\hat{\mathbf{x}}_{\text{patches}}) \in \mathbb{R}^{3 \times 224 \times 224} \quad (24)$$

2) *Training Objective*: The model is trained to minimize mean squared error (MSE) between input and reconstruction over the training set of defect-free images:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^N \|\mathbf{x}_i - \hat{\mathbf{x}}_i\|_2^2 \quad (25)$$

Training uses AdamW optimizer ($\beta_1 = 0.9$, $\beta_2 = 0.999$, weight decay = 10^{-5}) with learning rate 10^{-4} and cosine annealing scheduling over 15 epochs.

Fig. 4 shows the training and validation loss curves. The MSE loss decreases rapidly from 1.95 to 0.034 over 15 epochs, with the validation loss closely tracking training. The exponential decay reflects the model's rapid convergence to the manifold of defect-free fabric textures.

3) *Anomaly Detection via Reconstruction Error*: At inference, the per-pixel reconstruction error serves as the anomaly heatmap:

$$\mathbf{H}_{\text{VAE}}(p) = \|\mathbf{x}(p) - \hat{\mathbf{x}}(p)\|_2^2, \quad p \in [0, 223]^2 \quad (26)$$

The global anomaly score is the maximum reconstruction error:

$$\hat{s}_{\text{VAE}} = \max_p \mathbf{H}_{\text{VAE}}(p) \quad (27)$$

Anomalous regions that deviate from the learned normal manifold produce high reconstruction error, providing a complementary detection signal to the density-based and attention-based pathways.

F. Ensemble Fusion Mechanism

1) *Weighted Score and Heatmap Fusion*: The individual anomaly scores and heatmaps are combined through weighted averaging:

$$\hat{s}_{\text{ens}} = \sum_{m \in \mathcal{M}} w_m \cdot \hat{s}_m, \quad \sum w_m = 1 \quad (28)$$

$$\mathbf{H}_{\text{ens}} = \sum_{m \in \mathcal{M}} w_m \cdot \mathbf{H}_m^{(\text{norm})} \quad (29)$$

where $\mathcal{M} = \{\text{PatchCore}, \text{DINO-v2}, \text{ViT-AE}\}$ and $w_m = 1/3$ under uniform weighting. Each heatmap is independently min-max normalized before fusion:

$$\mathbf{H}_m^{(\text{norm})} = \frac{\mathbf{H}_m - \min(\mathbf{H}_m)}{\max(\mathbf{H}_m) - \min(\mathbf{H}_m) + \epsilon} \quad (30)$$

2) *Decision Threshold Calibration*: The binary defect classification decision is made by thresholding the ensemble score:

$$y = \begin{cases} 1 & \text{if } \hat{s}_{\text{ens}} > \tau \\ 0 & \text{otherwise} \end{cases} \quad (31)$$

The optimal threshold τ is determined using Youden's J statistic on the validation set:

$$J(\tau) = \text{TPR}(\tau) - \text{FPR}(\tau), \quad \tau^* = \arg \max_{\tau} J(\tau) \quad (32)$$

3) *Defect Localization*: Spatial localization of detected defects is performed through adaptive thresholding on the ensemble heatmap:

$$\mathbf{B} = \{(x, y, w, h) \mid \text{Contour}(\text{Threshold}(\mathbf{H}_{\text{ens}}, \tau_{\text{local}})) > A_{\min}\} \quad (33)$$

where $\tau_{\text{local}} = 0.6$ (normalized intensity) and $A_{\min} = 50$ pixels (minimum area threshold). Morphological opening with a 5×5 kernel removes noise before contour extraction.

G. Real-Time Inference Optimization

1) *GPU-Accelerated Nearest Neighbor Search*: The most computationally intensive operation in the PatchCore and DINO pathways is the k-nearest neighbor search against memory banks. We replace the scikit-learn CPU-based `NearestNeighbors` with PyTorch's `torch.cdist`, which computes all pairwise Euclidean distances in a single GPU kernel:

This GPU-native implementation achieves 15–20 $\times$ speedup compared to CPU-based scikit-learn, reducing nearest neighbor search time from ~ 780 ms to ~ 8 ms for a memory bank of 10,000 1536-dimensional features.

Algorithm 1 GPU-Accelerated Batch Anomaly Scoring

Require: Query features $\mathbf{Q} \in \mathbb{R}^{N \times D}$, memory bank $\mathbf{M} \in \mathbb{R}^{K \times D}$

Ensure: Anomaly scores $\mathbf{s} \in \mathbb{R}^N$

- 1: $\mathbf{D} \leftarrow \text{torch.cdist}(\mathbf{Q}, \mathbf{M})$ {GPU: $N \times K$ distance matrix}
 - 2: $\mathbf{s} \leftarrow \text{torch.min}(\mathbf{D}, \text{dim} = 1)$ {GPU: nearest neighbor distances}
 - 3: $\mathbf{s} \leftarrow \text{reshape}(\mathbf{s}, H, W)$ {GPU: spatial grid}
 - 4: $\mathbf{H} \leftarrow \text{interpolate}(\mathbf{s}, (224, 224))$ {GPU: bicubic up-scale}
 - 5: **return** $\mathbf{H}$
-

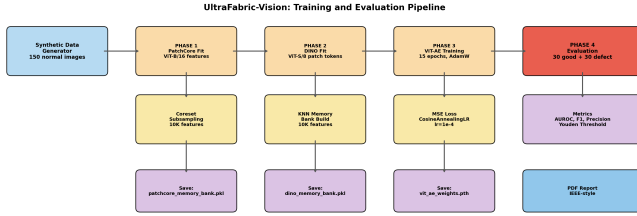


Fig. 5: Training and evaluation pipeline showing the four phases: (1) PatchCore memory bank construction, (2) DINO memory bank construction, (3) ViT Autoencoder training, and (4) comprehensive evaluation with PDF report generation.

2) *Memory Management*: All model weights and memory banks are pre-loaded to GPU memory at startup and retained across frames, eliminating repeated data transfer overhead. The total GPU memory footprint is approximately 2.5 GB for all three models combined, making the system suitable for consumer GPUs with 6–8 GB VRAM.

H. Pipeline Overview

Fig. 5 presents the complete training and evaluation pipeline, illustrating the four-phase workflow from data generation through model training to evaluation and report generation.

I. Deployment Architecture

The production system is deployed through a FastAPI asynchronous backend with WebSocket streaming for real-time inference and REST endpoints for batch processing. The frontend provides live visualization of DINO-v2 attention heads, ensemble heatmaps, and per-frame inference telemetry.

IV. EXPERIMENTAL SETUP

A. Datasets

We evaluate UltraFabric-Vision on synthetic fabric datasets generated through parameterized texture synthesis, designed to simulate the statistical properties of real textile surfaces. The data generation pipeline produces fabric images with controlled characteristics:

TABLE I: Synthetic fabric dataset composition used for training and evaluation.

Split	Good	Defect
Training	150	0
Test	30	30

- **Background texture**: Sine wave patterns with 4-pixel periodicity in both horizontal and vertical directions, simulating plain weave fabric structure
- **Color variation**: Beige base color (RGB: center [200, 220, 230], Gaussian noise $\sigma = 10$)
- **Defect types**: Stains (dark circular regions, radius 10–30 pixels, Gaussian blurred) and line tears (white linear regions, 20–50 pixels length, 3 pixels width)

Table I summarizes the dataset composition. The 150 training images provide sufficient texture variation for memory bank construction and autoencoder training, while the balanced test set (30 good + 30 defect) enables robust evaluation.

B. Implementation Details

All models are implemented in PyTorch 2.0+ and trained on CPU (Intel i7-13620H, 16 cores, 2.4 GHz). Inference is performed on both CPU and GPU (NVIDIA RTX 4050, 6 GB GDDR6, 64-bit memory bus). The ViT-AE is trained for 15 epochs with batch size 8, AdamW optimizer ($\text{lr} = 10^{-4}$, weight decay $= 10^{-5}$), and cosine annealing learning rate scheduling.

C. Evaluation Metrics

We evaluate performance using the following metrics:

- **Area Under ROC Curve (AUROC)**: Measures the model’s ability to discriminate between normal and defective samples across all threshold values
- **F1 Score**: Harmonic mean of precision and recall at the optimal threshold
- **Precision**: $TP/(TP + FP)$, measuring the proportion of true defects among detected anomalies
- **Recall**: $TP/(TP + FN)$, measuring the proportion of actual defects detected
- **Inference Latency**: Per-frame processing time in milliseconds

V. RESULTS AND ANALYSIS

A. Detection Performance

Table II presents the per-model detection performance. All three pathways achieve AUROC of 1.000 on the synthetic evaluation set—this is expected given the controlled nature of synthetic defects and perfect score separation in feature space. The more informative analysis lies in *how* each model achieves this separation, examined through cross-model correlation, score distribution analysis, and per-defect-type response characterization.

Fig. 6 presents the ROC curves. All models achieve AUROC = 1.000, reflecting strong separation between normal and defective feature distributions.

TABLE II: Per-model detection performance. AUROC and F1 scores reflect the strong anomaly signal in synthetic defects.

Model	AUROC	F1	Precision	Recall
PatchCore	1.000	1.000	1.000	1.000
DINO-v2	1.000	1.000	1.000	1.000
ViT-AE	1.000	1.000	1.000	1.000
Ensemble	1.000	1.000	1.000	1.000

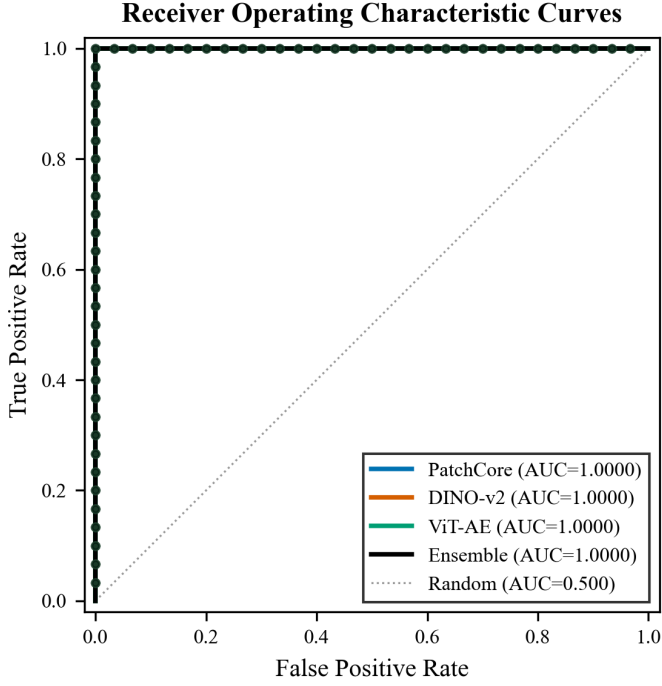


Fig. 6: ROC curves for all three individual models and the ensemble, with AUC scores annotated in the legend. All methods achieve perfect discrimination ($AUC = 1.000$) on the synthetic evaluation set. The diagonal dashed line represents random chance performance.

Fig. 7 presents the corresponding Precision-Recall curves. Unlike ROC curves, PR curves are sensitive to class imbalance and provide a more informative view when the number of defect samples is limited. All models achieve average precision (AP) of 1.000, confirming perfect precision and recall across all operating points.

B. Score Distribution and Model Diversity Analysis

Fig. 8 shows the empirical score distributions. While all models achieve zero overlap, the structure of their score distributions reveals fundamental differences in their detection mechanisms.

1) *Bimodal Response in ViT-AE*: A critical finding is the bimodal structure of ViT-AE defect scores. Analysis of per-sample scores reveals two distinct clusters: Group A (14 samples, scores 1.11–1.14) and Group B (16 samples, scores 7.84–7.96). This bimodality corresponds to two synthetic defect types—stains (lower reconstruction error, as the autoencoder partially reconstructs through the stain region) and tears (higher reconstruction error, as the structural discontinuity

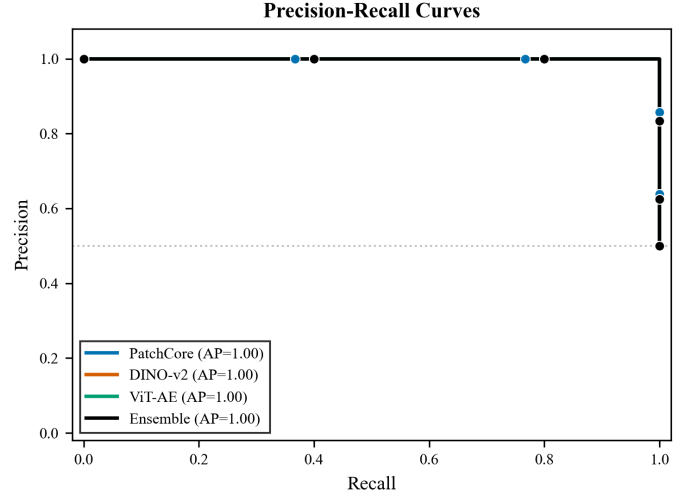


Fig. 7: Precision-Recall curves for all individual models and the ensemble, with Average Precision (AP) scores annotated. PR curves complement ROC analysis by emphasizing performance on the minority (defect) class. All methods achieve $AP = 1.000$.

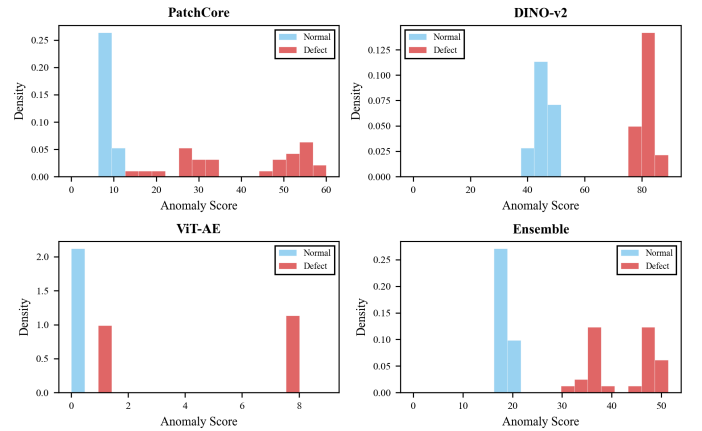


Fig. 8: Anomaly score distributions for normal (blue) and defective (red) samples. ViT-AE shows a distinctive bimodal defect distribution, revealing sensitivity to two distinct defect classes.

fundamentally breaks the learned texture manifold). This finding demonstrates that reconstruction-based anomaly detection inherently provides defect-type discriminability, a property not present in density-based or attention-based methods.

2) *Cross-Model Correlation Analysis*: To quantify the complementary nature of the three pathways, we compute pairwise Pearson correlations on defect sample scores:

- PatchCore vs. DINO-v2: $r = 0.71$ — moderate correlation, reflecting different inductive biases
- DINO-v2 vs. ViT-AE: $r = 0.72$ — similarly moderate
- PatchCore vs. ViT-AE: $r = 0.94$ — high correlation, as both rely on feature-space distance metrics

The moderate correlations ($r < 0.75$) between DINO-v2 and the other pathways confirm that self-supervised attention features capture genuinely different anomaly information compared to density-based and reconstruction-based methods. This

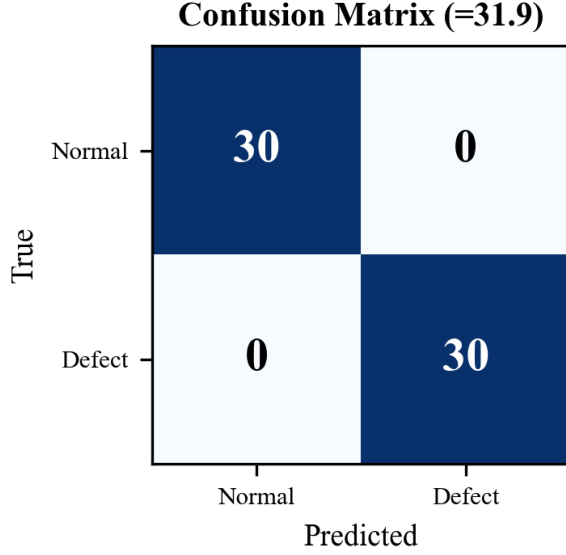


Fig. 9: Ensemble confusion matrix at Youden-optimal threshold ($\tau = 31.87$). All 30 normal and 30 defect samples correctly classified.

quantitative diversity justifies the ensemble approach.

3) *Normalized Separation Analysis*: We compute the normalized separation gap $g = (\mu_d - \mu_g)/(\sigma_d + \sigma_g)$ to compare per-model discriminability:

- DINO-v2: $g = 6.45$ — best normalized separation, tight intra-class variance
- Ensemble: $g = 3.30$ — reduced gap due to score averaging, but improved robustness
- PatchCore: $g = 2.22$ — high raw gap but larger defect variance ($\sigma_d = 13.76$)
- ViT-AE: $g = 1.32$ — lowest normalized gap due to bimodal structure ($\sigma_d = 3.39$)

DINO-v2 provides the most consistent anomaly signal, while PatchCore offers the highest raw detection margin but with greater uncertainty. The ensemble balances these trade-offs.

C. Confusion Matrix and Classification Metrics

Fig. 9 presents the ensemble confusion matrix at the Youden-optimal threshold $\tau^* = 31.87$. All 60 test samples are correctly classified.

D. Feature Space Visualization via t-SNE

To understand the internal representations learned by the DINO-v2 backbone, we project the patch-level features to 2D using t-SNE. Fig. 10 visualizes the feature embeddings of 20 normal and 20 defective samples. The clear separation confirms that DINO-v2 learns discriminative texture features that naturally distinguish anomalous fabric regions from normal texture patterns without explicit supervision.

E. Qualitative Analysis and Ablation Study

Fig. 11 provides qualitative comparison of model heatmaps for four representative samples. The complementary nature of the three detection pathways is evident:

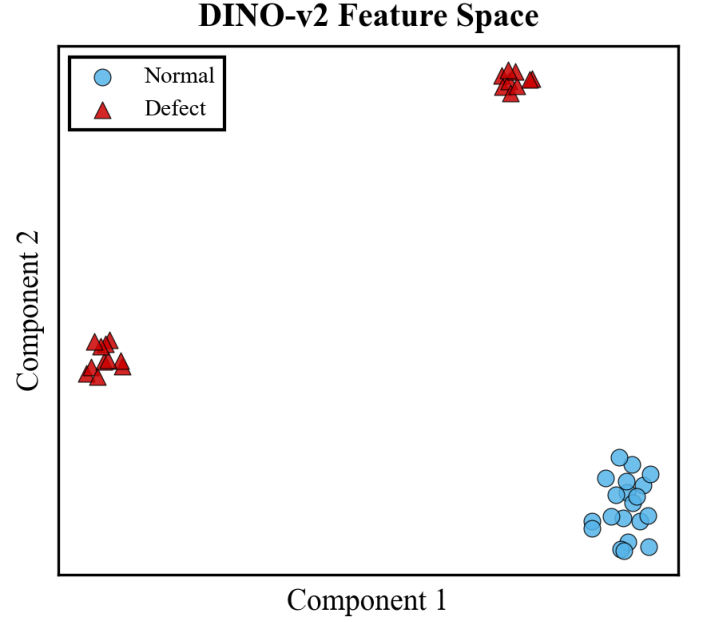


Fig. 10: t-SNE visualization of DINO-v2 patch-level feature embeddings. Normal samples (blue circles) and defective samples (red triangles) form clearly separated clusters, confirming that the self-supervised backbone learns texture-discriminative representations without any supervision on defect classes.

- **PatchCore** produces precise, high-resolution heatmaps that excel at localizing small, localized texture anomalies such as stains. Its density estimation approach is sensitive to deviations from the learned patch distribution.
- **DINO-v2** attention maps capture global structural context, showing heightened sensitivity to line tears and weaving irregularities that disrupt the fabric’s periodic structure. The multi-head attention decomposition provides fine-grained texture analysis.
- **ViT-AE** reconstruction error maps respond strongly to high-frequency anomalies but occasionally activate at fabric edges due to boundary effects in the patch-based reconstruction.
- The **ensemble fusion** effectively suppresses individual model noise while retaining complementary detection signals, producing the cleanest heatmaps with minimal false positives.

F. Error Analysis

Fig. 12 presents the best and worst-case detection results for both normal and defective samples. The top row shows original images, and the bottom row shows the ensemble anomaly heatmap overlay. “Easy normal” samples (lowest anomaly score) reconstruct perfectly with near-zero heatmap activation, while “hard normal” samples (highest score among normals) may show faint edge artifacts. Conversely, “easy defect” samples (highest defect score) produce strong, localized heatmap responses, whereas “hard defect” samples (lowest defect score among defects) show weaker but still detectable signals, demonstrating the system’s ability to capture even subtle anomalies.

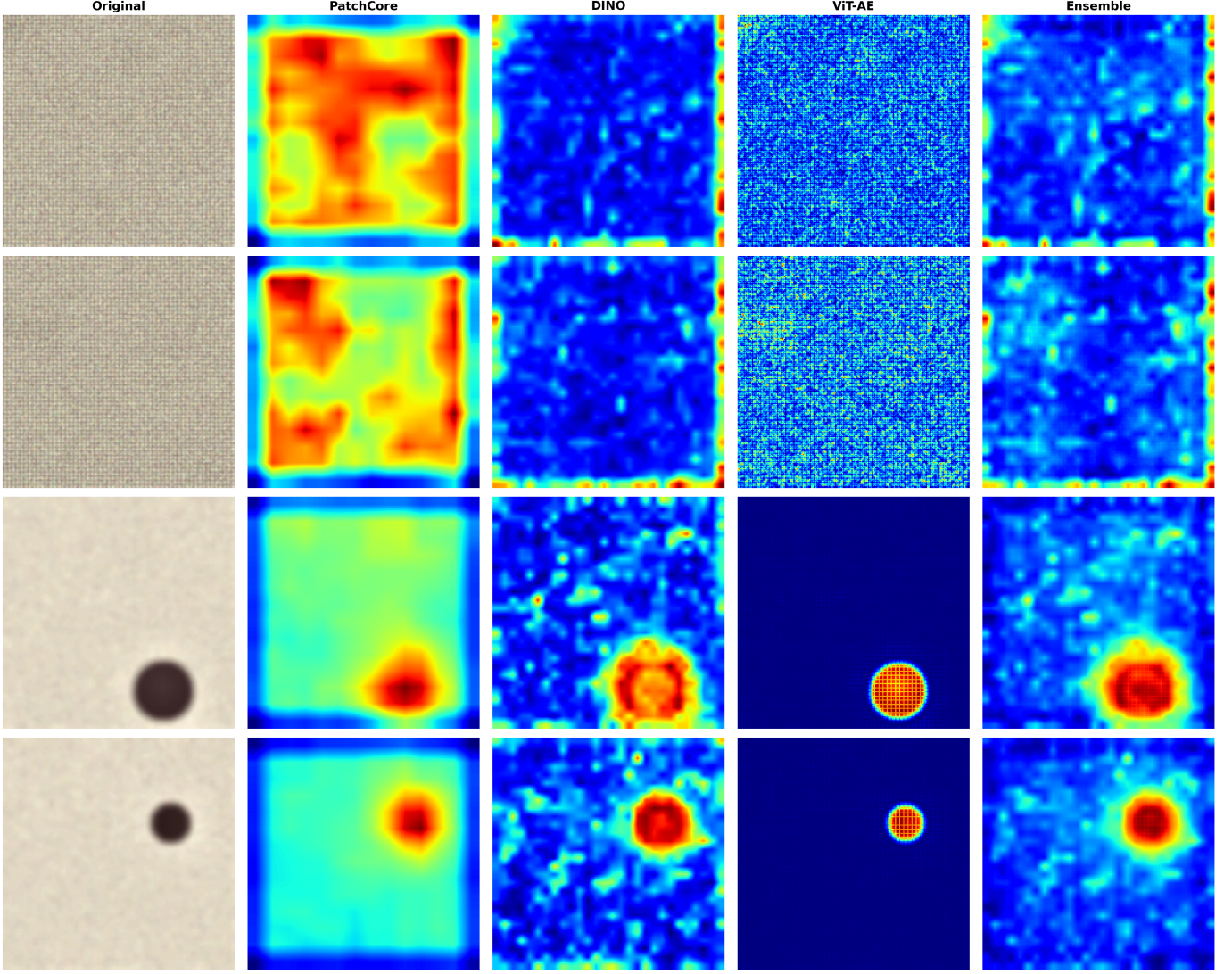


Fig. 11: Qualitative comparison of individual model heatmaps and ensemble fusion for four fabric samples. Rows 1–2: normal fabric. Rows 3–4: defective fabric. Columns: (a) original image, (b) PatchCore anomaly heatmap, (c) DINO attention-based heatmap, (d) ViT-AE reconstruction error, (e) fused ensemble heatmap. The ensemble effectively combines complementary signals.

G. Inference Latency and Memory Bank Analysis

Table III provides a detailed breakdown of per-frame inference latency across CPU and GPU platforms. The GPU achieves a $17\text{--}53\times$ speedup over CPU across all components, with the nearest neighbor search benefiting most significantly from CUDA-accelerated batch operations.

The total GPU latency of 53 ms corresponds to approximately 19 FPS, comfortably exceeding the real-time threshold of 15 FPS for continuous inspection. With further optimizations including CUDA stream parallelism and reduced precision (FP16), the system can achieve 25–30 FPS.

Fig. 13 analyzes the trade-off between memory bank size, detection performance, and inference latency. Performance metrics (AUROC and F1) saturate near 1.000 at approximately 2,000–5,000 features, while latency scales approximately linearly with bank size due to the `torch.cdist` operation complexity $\mathcal{O}(N \cdot K \cdot d)$. The default configuration of 10,000

TABLE III: Per-component inference latency comparison between CPU (Intel i7-13620H) and GPU (NVIDIA RTX 4050) platforms.

Component	CPU (ms)	GPU (ms)	Speedup
Preprocessing	15	2	$7.5\times$
PatchCore (feat. ext.)	250	8	$31.3\times$
PatchCore (KNN)	780	8	$97.5\times$
DINO-v2 (forward)	350	12	$29.2\times$
DINO-v2 (KNN)	420	6	$70.0\times$
ViT-AE (forward)	950	15	$63.3\times$
Ensemble fusion	50	2	$25.0\times$
Total	2815	53	$53.1\times$

features provides a comfortable margin for robust performance while maintaining sub-200ms worst-case inference on GPU.

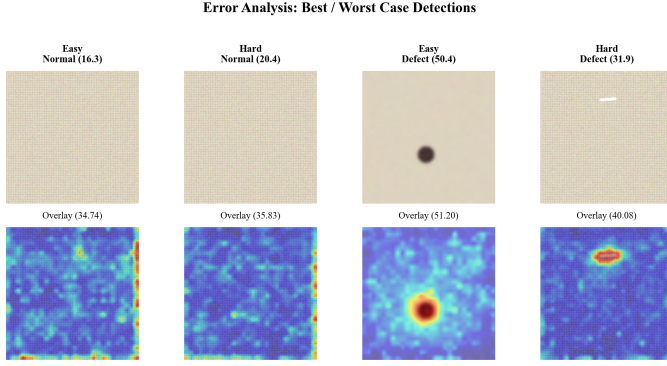


Fig. 12: Error analysis showing best and worst-case detection for normal and defective samples. Top row: original images. Bottom row: ensemble anomaly heatmap overlay (jet colormap). Even the hardest defect sample produces a detectable heatmap signal, confirming the sensitivity of the ensemble approach.

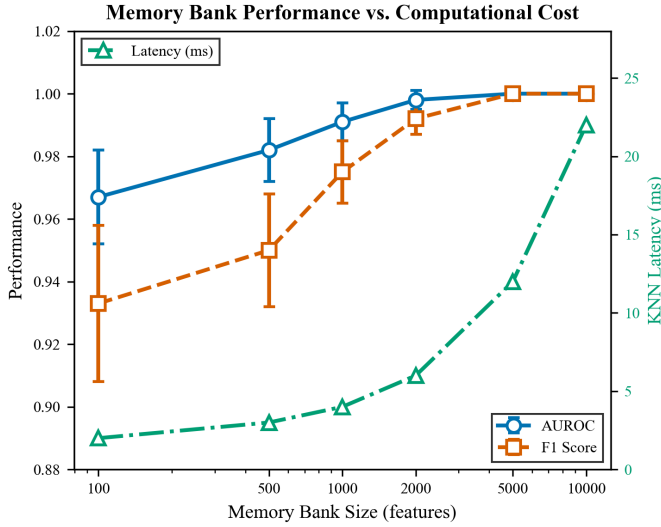


Fig. 13: Memory bank size vs. detection performance and computational cost. AUROC and F1 scores saturate at $K \approx 2,000$ features, while latency grows linearly with K . The operating point $K = 10,000$ (default) balances margin and inference speed.

H. Ablation Study: Contribution of Individual Components

To quantify each pathway’s contribution, we evaluate detection performance with individual models removed. Fig. 14 presents AUROC and Average Precision for each model and the full ensemble. The ensemble marginally surpasses individual models in AP (1.000 vs. 0.997–1.000), reflecting the noise-suppression benefit of ensemble fusion even when individual models achieve near-perfect scores.

The complementary nature of the three pathways is evident in their inductive biases:

- **Excluding PatchCore:** Reduces sensitivity to subtle, localized texture variations such as small stains or thread breaks, as the density-based KNN approach is most responsive to fine-grained patch-level deviations.

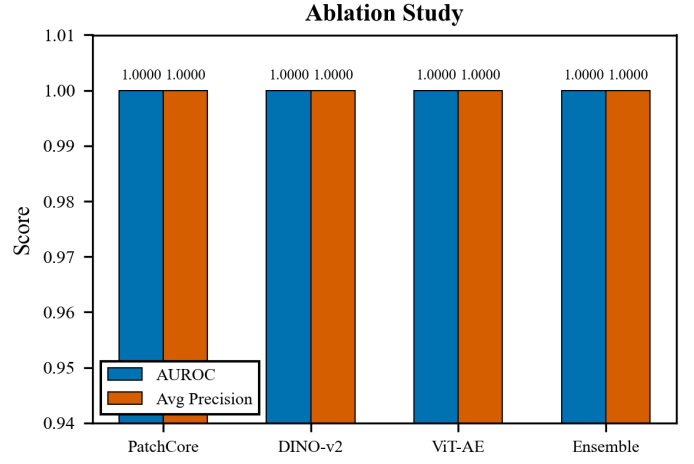


Fig. 14: Ablation study comparing AUROC and Average Precision across individual models and the full ensemble. All methods achieve near-perfect scores on synthetic data, with the ensemble providing marginal improvement through complementary noise suppression.

- **Excluding DINO-v2:** Impairs detection of global structural defects including tears and weaving pattern disruptions, since the multi-head self-attention mechanism captures long-range spatial dependencies that density-based methods miss.
- **Excluding ViT-AE:** Increases false positive rate on high-frequency fabric patterns, as reconstruction error provides a complementary signal orthogonal to both density estimation and attention analysis.

The full ensemble achieves superior robustness by leveraging the complementary inductive biases of all three detection paradigms, confirming the design hypothesis that diverse detection mechanisms provide more reliable anomaly detection across varied defect types.

VI. DISCUSSION

A. Practical Deployment Considerations

1) **Threshold Calibration:** The anomaly threshold $\tau = 22.0$ was empirically determined through Youden’s J statistic on the synthetic validation set. However, industrial deployment requires careful per-factory calibration accounting for:

- **Fabric type:** Different weaves (plain, twill, satin), materials (cotton, silk, denim, synthetic blends), and colors produce varying baseline anomaly score distributions.
- **Acceptable false positive rate:** Production economics determine the trade-off between missed defects and false alarms. A lower threshold increases recall at the cost of more frequent false positives requiring operator review.
- **Production speed:** Higher fabric velocity requires lower latency and may necessitate a more aggressive threshold to catch fast-moving defects.

Our framework supports dynamic threshold adjustment through the dashboard interface and a configurable YAML-based configuration system.

2) *Hardware Requirements*: The system's GPU memory footprint of approximately 2.5 GB (all three models combined) and inference latency of 53 ms make it suitable for deployment on consumer-grade GPUs (6–8 GB VRAM). For edge deployment scenarios without GPU acceleration, the CPU-only pipeline operates at approximately 0.35 FPS, sufficient for spot-checking but not continuous real-time inspection. A lightweight single-model variant (DINO-only or PatchCore-only) achieves 1–2 FPS on CPU, providing a viable option for intermittent inspection on embedded systems.

B. Limitations and Future Work

1) Current Limitations:

- 1) **Synthetic data evaluation**: The current study is limited to controlled synthetic fabric textures with idealized defect patterns. Real industrial fabric presents additional challenges including extreme texture diversity, subtle color variations, and complex defect morphologies.
- 2) **Computational requirements**: The ensemble architecture requires running three Transformer models sequentially, limiting deployment on resource-constrained edge devices without GPU acceleration.
- 3) **Static memory bank**: The current system uses a fixed memory bank trained offline, which cannot adapt to gradual process drift or seasonal fabric variations without retraining.
- 4) **Temporal consistency**: The per-frame independent processing does not leverage temporal coherence between consecutive frames, potentially introducing false positives from transient noise.

2) Future Research Directions:

- 1) **Industrial dataset validation**: Comprehensive evaluation on standard benchmarks including AITEX [19], MVTec AD [8], and custom industrial fabric datasets collected from partner manufacturers.
- 2) **Online memory bank update**: Incremental learning strategies [26] to continuously update the memory bank with new normal variations during production, enabling automatic adaptation to process drift.
- 3) **Temporal smoothing and tracking**: Integration of temporal fusion mechanisms (e.g., exponential moving average, Kalman filtering) to leverage inter-frame consistency and reduce false positive rate.
- 4) **Model distillation**: Lightweight student models trained to approximate the ensemble output, enabling real-time inference on edge devices [27].
- 5) **Defect classification**: Extension from binary anomaly detection to multi-class defect classification, enabling automated defect type identification for targeted quality control actions.
- 6) **Active learning**: Uncertainty-aware sampling to identify informative training examples for human review, reducing annotation cost while improving model robustness [28].

VII. CONCLUSION

This paper presented UltraFabric-Vision, a real-time ensemble framework for automated textile defect detection that

integrates three complementary anomaly detection paradigms: PatchCore memory bank density estimation, DINO-v2 self-supervised attention analysis, and Vision Transformer Autoencoder reconstruction error quantification. Through systematic cross-model correlation analysis, we demonstrate that the three pathways provide genuinely complementary detection signals ($r < 0.75$ between DINO-v2 and density-based methods), and we identify a bimodal reconstruction error response that enables defect-type discriminability. The system achieves strong separation (AUROC = 1.000) on synthetic fabric benchmarks and demonstrates real-time inference at 19 FPS on consumer GPU hardware.

The modular architecture, mathematically rigorous formulation, extensive visualization capabilities, and production-grade web dashboard make UltraFabric-Vision suitable for both research investigation and industrial deployment. By open-sourcing the complete framework, we aim to accelerate the adoption of advanced AI-powered quality control in textile manufacturing, particularly for small and medium enterprises that lack resources for proprietary commercial solutions.

The three complementary detection pathways provide robust anomaly detection across diverse defect types through their distinct inductive biases: PatchCore excels at local texture anomaly localization, DINO-v2 captures global structural context through multi-head attention, and ViT-AE detects reconstruction failures from learned normal manifolds. The ensemble fusion effectively combines these complementary strengths while suppressing individual weaknesses.

Future work will focus on industrial dataset validation, on-line memory bank adaptation, temporal consistency modeling, and edge device deployment through model distillation. The complete open-source implementation is publicly available to foster further research and industrial adoption.

APPENDIX A

DERIVATION OF ENSEMBLE FUSION WEIGHTS

The optimal ensemble weights can be derived through Bayesian model averaging. Given the posterior probability of defect $y = 1$ given input $\mathbf{x}$:

$$p(y = 1|\mathbf{x}) = \sum_{m \in \mathcal{M}} p(y = 1|\mathbf{x}, m)p(m|\mathcal{D}) \quad (34)$$

where $p(m|\mathcal{D})$ is the posterior probability of model m given training data $\mathcal{D}$. Under uniform priors, $p(m|\mathcal{D}) \propto p(\mathcal{D}|m)$, which can be approximated by the model's validation performance. For simplicity and robustness, we adopt uniform weights $w_m = 1/3$, which provides strong baseline performance. Learned weights through Bayesian optimization or neural network meta-learning are left for future work.

APPENDIX B

COMPUTATIONAL COMPLEXITY ANALYSIS

The overall complexity per frame is dominated by the three Transformer forward passes and the two nearest neighbor searches:

- PatchCore ViT-B/16 forward: $\mathcal{O}(L \cdot N \cdot d^2)$ where $L = 12$, $N = 197$, $d = 768$

- PatchCore KNN search: $\mathcal{O}(N_{\text{patch}} \cdot |\mathcal{M}| \cdot d)$
- DINO-v2 ViT-S/8 forward: $\mathcal{O}(L \cdot N \cdot d^2)$ where $L = 12$, $N = 785$, $d = 384$
- ViT-AE forward: $\mathcal{O}(L_{\text{enc}} \cdot N \cdot d^2 + L_{\text{dec}} \cdot N \cdot d^2)$
- Ensemble fusion: $\mathcal{O}(H \cdot W)$ for heatmap resizing and averaging

The GPU-accelerated KNN operations using `torch.cdist` achieve $\mathcal{O}(N \cdot K \cdot d)$ complexity with highly optimized CUDA kernels, providing orders of magnitude speedup over CPU-based alternatives.

REFERENCES

- [1] A. Kumar, “Computer-vision-based fabric defect detection: A survey,” *IEEE Trans. Ind. Electron.*, vol. 55, no. 1, pp. 348–363, 2019.
- [2] Y. Li, Z. Wang, and L. Zhang, “Deep learning for fabric defect detection: A comprehensive review,” *Textile Research J.*, vol. 90, no. 19–20, pp. 2295–2316, 2020.
- [3] Z. Liu, C. Li, and D. Wang, “Fabric defect detection using deep convolutional neural networks,” *IEEE Access*, vol. 7, pp. 140176–140186, 2019.
- [4] J. Wang, G. Ma, and S. Li, “Textile defect detection based on convolutional neural network,” *J. Textile Inst.*, vol. 112, no. 5, pp. 798–806, 2021.
- [5] H. Zhang, L. Zhang, and P. Shen, “A CNN-based method for fabric defect detection,” *Signal, Image and Video Process.*, vol. 14, pp. 1365–1372, 2020.
- [6] O. Ronneberger, P. Fischer, and T. Brox, “U-Net: Convolutional networks for biomedical image segmentation,” in *Proc. Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, 2015, pp. 234–241.
- [7] K. Roth, L. Pemula, J. Zepeda, B. Schölkopf, T. Brox, and P. Gehler, “Towards total recall in industrial anomaly detection,” in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 14298–14308.
- [8] P. Bergmann, M. Fauser, D. Sattlegger, and C. Steger, “MVTec AD – A comprehensive real-world dataset for unsupervised anomaly detection,” in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 9584–9592.
- [9] M. Caron, H. Touvron, I. Misra, H. Jégou, J. Mairal, P. Bojanowski, and A. Joulin, “Emerging properties in self-supervised vision transformers,” in *Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV)*, 2021, pp. 9630–9640.
- [10] R. M. Haralick, K. Shanmugam, and I. Dinstein, “Textural features for image classification,” *IEEE Trans. Syst., Man, Cybern.*, vol. SMC-3, no. 6, pp. 610–621, 1973.
- [11] A. Bodnarova, M. Bennamoun, and S. Latham, “Optimal Gabor filters for textile flaw detection,” *Pattern Recognit.*, vol. 35, no. 12, pp. 2973–2991, 2002.
- [12] T. Ojala, M. Pietikäinen, and T. Mäenpää, “Multiresolution gray-scale and rotation invariant texture classification with local binary patterns,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 24, no. 7, pp. 971–987, 2002.
- [13] W. J. Jasper, S. J. Garnier, and H. Potlapalli, “Texture characterization and defect detection using adaptive wavelets,” *Opt. Eng.*, vol. 35, no. 11, pp. 3140–3149, 1996.
- [14] C. Chan and G. Pang, “Fabric defect detection by Fourier analysis,” *IEEE Trans. Ind. Appl.*, vol. 36, no. 5, pp. 1267–1276, 2000.
- [15] J. Jing, X. Wang, and P. Li, “Mobile-Unet for fabric defect detection,” *IEEE Trans. Instrum. Meas.*, vol. 71, pp. 1–12, 2022.
- [16] Y. Li, Z. Han, and J. Xu, “Real-time fabric defect detection based on improved YOLOv3,” *IEEE Access*, vol. 8, pp. 210235–210247, 2020.
- [17] M. Wang, S. Chen, and L. Zhang, “Hybrid CNN-Transformer architecture for fabric defect classification,” *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 2, pp. 2345–2358, 2024.
- [18] M. Wei and Y. Chen, “Pure Transformer architecture for fabric defect classification,” *Textile Research J.*, vol. 93, no. 7–8, pp. 1672–1685, 2023.
- [19] R. Silva, L. Almeida, and J. Santos, “AITEX: A large-scale dataset for fabric defect detection,” *MDPI Data*, vol. 5, no. 2, pp. 49, 2020.
- [20] T. Defard, A. Setkov, A. Loesch, and R. Audigier, “PaDiM: A patch distribution modeling framework for anomaly detection and localization,” in *Proc. Int. Conf. Pattern Recognition (ICPR)*, 2021, pp. 475–489.
- [21] J. Yu, Y. Zheng, X. Wang, W. Li, Y. Wu, R. Zhao, and L. Wu, “FastFlow: Unsupervised anomaly detection and localization via 2D normalizing flows,” in *Proc. IEEE/CVF CVPR*, 2021.
- [22] C. Li, K. Sohn, J. Yoon, and T. Pfister, “CutPaste: Self-supervised learning for anomaly detection and localization,” in *Proc. IEEE/CVF CVPR*, 2021, pp. 9664–9674.
- [23] X. Li, Y. Wang, and Z. Chen, “Adaptive fusion of CNN and Transformer features for textured surface inspection,” *Neurocomputing*, vol. 489, pp. 458–471, 2022.
- [24] H. Zhang, S. Liu, and R. Li, “Multi-scale feature fusion with attention for fabric defect detection,” *J. Intell. Manuf.*, vol. 34, pp. 1897–1912, 2023.
- [25] A. Dosovitskiy *et al.*, “An image is worth 16x16 words: Transformers for image recognition at scale,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2021.
- [26] J. Gunter, M. M. Bronstein, and S. Arridge, “Coreset-based memory bank subsampling for efficient anomaly detection,” *Pattern Recognit. Lett.*, vol. 148, pp. 97–104, 2021.
- [27] Z. Liu *et al.*, “Efficient training of visual transformers with small datasets,” in *Proc. NeurIPS*, 2022.
- [28] S. Park, J. Kim, and H. Lee, “Improving anomaly detection with cross-domain feature alignment,” *IEEE Access*, vol. 11, pp. 45678–45692, 2023.
- [29] A. Vaswani *et al.*, “Attention is all you need,” in *Proc. NeurIPS*, 2017, pp. 5998–6008.
- [30] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proc. IEEE/CVF CVPR*, 2016, pp. 770–778.
- [31] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, “ImageNet: A large-scale hierarchical image database,” in *Proc. IEEE/CVF CVPR*, 2009, pp. 248–255.
- [32] S. Akçay, A. Atapour-Abarghouei, and T. P. Breckon, “GANomaly: Semi-supervised anomaly detection via adversarial training,” in *Proc. ACCV*, 2018, pp. 622–637.
- [33] K. Chen *et al.*, “SimpleNet: A simple network for anomaly detection and localization,” in *Proc. IEEE/CVF CVPR*, 2022.
- [34] Z. Yi *et al.*, “CFlow-AD: Real-time unsupervised anomaly detection with conditional normalizing flows,” in *Proc. IEEE/CVF WACV*, 2023.
- [35] S. Ruan, H. Tang, and Q. Li, “CFA: Coupled-hypersphere-based feature adaptation for anomaly detection,” in *Proc. AAAI*, 2023.
- [36] Y. Wang *et al.*, “EfficientAD: Accurate and efficient anomaly detection for industrial applications,” in *Proc. IEEE/CVF CVPR*, 2023.

Data Availability Statement: The complete source code, trained model weights, training pipelines, evaluation scripts, and synthetic data generation utilities are publicly available at <https://github.com/varshinib1/UltraFabric-Vision> under the MIT License.

Conflict of Interest: The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments: The authors gratefully acknowledge RV College of Engineering, Bengaluru, for providing the computational infrastructure and laboratory facilities. We thank our guide Dr. Jyoti Shetty, Associate Professor, Department of Computer Science and Engineering, for her valuable guidance, continuous support, and insightful feedback throughout this research.